



Blockchain and Cryptocurrency

Project: Assetry — Escrow-Based Digital Goods Marketplace

Team Members:

Muhammad Ali Jauhar (22I-1705)

Abdullah bin Umar (22I-1690)

Muhammad Abdullah Khan Saadi (22I-1652)

Section: B

Submitted To: Dr Muhammad Asim

Table of Contents

1. Introduction
2. Use Case Rationale
3. System Architecture
4. Smart Contract Technical Specifications
5. Backend Architecture
6. Frontend Architecture
7. Database Design
8. Authentication Flow
9. File Storage
10. Transaction Lifecycle
11. Workload Distribution
12. Conclusion

1. Introduction

Assetry is a decentralized marketplace for buying and selling digital goods — including software, digital art, music, ebooks, and templates. The platform leverages blockchain technology to implement a trustless escrow system, where payments are held in a smart contract and only released to the seller after the buyer explicitly confirms successful delivery of the purchased asset.

The core problem Assetry solves is the lack of trust in peer-to-peer digital goods transactions. In traditional platforms, buyers must trust that the seller will deliver after payment, and sellers must trust that the platform will forward their earnings. Assetry eliminates both risks by using a smart contract as a neutral, automated intermediary — one that cannot be manipulated by either party or the platform itself.

2. Use Case Rationale

2.1 Problem Statement

The digital goods market suffers from a fundamental trust problem:

- **For buyers:** There is no guarantee that a seller will deliver the promised asset after receiving payment. Chargebacks and disputes are handled by centralized platforms that may be slow, biased, or unavailable.
- **For sellers:** Centralized platforms take significant fees (typically 10–30%), hold funds for extended periods, and can freeze or ban accounts arbitrarily.
- **For both:** Centralized platforms are single points of failure — they can be hacked, shut down, or censored.

2.2 Why Blockchain?

Blockchain technology addresses these problems directly:

- **Trustless escrow:** A smart contract holds funds with rules that neither party can override. The contract code is public and immutable.
- **No intermediary fees:** Transactions happen directly between buyer and seller, with only gas fees paid to the network.
- **Wallet-based identity:** Users authenticate with their Ethereum wallet via SIWE (Sign-In with Ethereum), eliminating the need for usernames and passwords, and reducing the risk of account takeovers.
- **Transparent transaction history:** Every payment and release is recorded on the Sepolia blockchain and publicly verifiable on Etherscan.

2.3 Why Digital Goods?

Digital goods are an ideal use case for blockchain escrow because:

- Delivery is verifiable — the buyer either receives the file or they don't
- There is no physical logistics complexity
- The value of the goods is entirely in the file itself, making the escrow model clean and straightforward
- The market is large and underserved by trustless solutions

3. System Architecture

Assetry follows a three-tier architecture with an additional blockchain layer:



3.1 Tech Stack Summary

Layer	Technology
Frontend	Next.js 16, React 19
Wallet Integration	Wagmi v2, Viem
Authentication	SIWE (Sign-In with Ethereum), JWT
Backend	Node.js, Express 5
Database	PostgreSQL, Prisma ORM
File Storage	Multer (local filesystem)
Smart Contract	Solidity 0.8.24
Contract Development	Hardhat 3
Blockchain Network	Ethereum Sepolia Testnet

4. Smart Contract Technical Specifications

4.1 Contract Overview

- **Contract Name:** AssetryEscrow
- **Language:** Solidity 0.8.24
- **Network:** Ethereum Sepolia Testnet
- **Deployed Address:** 0xdCC00A8e85DED008937114ef7248068f1cD7ebcc
- **License:** MIT

4.2 Design Approach

Rather than deploying a new contract per purchase (which would be expensive in gas), Assetry uses a single shared contract that manages all escrow entries via a mapping. Each purchase is identified by a bytes32 key derived from the off-chain purchase UUID using keccak256.

mapping(bytes32 => EscrowEntry) public escrows;

This approach is gas-efficient and keeps the contract simple and auditable.

4.3 Data Structures

State Enum

```
enum State { AWAITING_PAYMENT, PENDING, CONFIRMED, REFUNDED }
```

State	Meaning
AWAITING_PAYMENT	Initial state (unused in current flow)
PENDING	ETH deposited, awaiting buyer confirmation
CONFIRMED	Buyer confirmed delivery, ETH released to seller

State	Meaning
REFUNDED	Buyer requested refund, ETH returned

EscrowEntry Struct

```
struct EscrowEntry {
    address payable buyer;
    address payable seller;
    uint256 amount;
    State state;
}
```

4.4 Functions

deposit(bytes32 purchaseld, address payable seller)

- **Visibility:** external payable
- **Called by:** Buyer
- **Purpose:** Locks ETH in the contract for a specific purchase
- **Validations:**
 - msg.value > 0 — must send ETH
 - Escrow entry must not already exist for this purchaseld
 - Seller address must be valid and not equal to buyer
- **Effect:** Creates an EscrowEntry with state PENDING
- **Emits:** Deposited(purchaseld, buyer, seller, amount)

confirmDelivery(bytes32 purchaseld)

- **Visibility:** external
- **Called by:** Buyer only
- **Purpose:** Releases locked ETH to the seller
- **Validations:**
 - Caller must be the original buyer
 - State must be PENDING
- **Effect:** Sets state to CONFIRMED, transfers ETH to seller
- **Emits:** Confirmed(purchaseld, seller, amount)

refund(bytes32 purchaseld)

- **Visibility:** external
- **Called by:** Buyer only
- **Purpose:** Returns locked ETH to the buyer (dispute)
- **Validations:**
 - Caller must be the original buyer
 - State must be PENDING
- **Effect:** Sets state to REFUNDED, transfers ETH back to buyer

- **Emits:** Refunded(purchaseId, buyer, amount)

getEscrow(bytes32 purchaseId)

- **Visibility:** external view
- **Purpose:** Read-only query of escrow state
- **Returns:** buyer, seller, amount, state

4.5 Events

event Deposited(bytes32 indexed purchaseId, address buyer, address seller, uint256 amount);

event Confirmed(bytes32 indexed purchaseId, address seller, uint256 amount);

event Refunded (bytes32 indexed purchaseId, address buyer, uint256 amount);

Events are indexed by purchaseId for efficient off-chain querying via Etherscan or event listeners.

4.6 Security Considerations

- **Re-entrancy:** The contract follows the checks-effects-interactions pattern — state is updated before ETH is transferred, preventing re-entrancy attacks.
- **Access control:** confirmDelivery and refund both verify msg.sender == e.buyer, ensuring only the buyer can trigger these actions.
- **Self-purchase prevention:** The deposit function rejects transactions where buyer and seller are the same address.
- **Duplicate prevention:** A purchaseId can only be used once — attempting to deposit for an existing escrow reverts.

4.7 Deployment

The contract was compiled and deployed using Hardhat 3 with the @nomicfoundation/hardhat-toolbox-mocha-ethers plugin. Deployment was performed via a script targeting the Sepolia network using an Alchemy RPC endpoint.

5. Backend Architecture

The backend is a RESTful API built with Node.js and Express 5, structured around four route modules.

5.1 Route Structure

Route	Description
POST /api/auth/nonce	Generates a random nonce for SIWE
POST /api/auth/verify	Verifies SIWE signature, returns JWT
GET /api/listings	Fetch all listings (with search/filter)
GET /api/listings/:id	Fetch a single listing
GET /api/listings/seller/:userId	Fetch listings by seller
POST /api/listings	Create listing with file upload

Route	Description
GET /api/listings/:id/download	Download product file (auth-gated)
DELETE /api/listings/:id	Delete listing and its file
POST /api/purchases	Create a purchase record
PATCH /api/purchases/:id/tx	Save on-chain transaction hash
PATCH /api/purchases/:id/confirm	Mark purchase as confirmed
PATCH /api/purchases/:id/dispute	Mark purchase as disputed
GET /api/purchases/my	Get buyer's purchases
GET /api/purchases/sales	Get seller's sales
GET /api/users/me	Get own profile
PATCH /api/users/me	Update profile
GET /api/users/:id	Get public profile

5.2 Middleware

- **middleware/auth.js:** Validates JWT from Authorization: Bearer header, attaches decoded user to req.user
- **middleware/upload.js:** Multer configuration supporting two upload fields — file (product, up to 100MB) and preview (image only)

5.3 File Download Security

Product files are not served publicly. The download route (GET /api/listings/:id/download) verifies that the requester is either: - The original seller, or - A buyer with a confirmed purchase status

Only then does the server stream the file using res.download().

6. Frontend Architecture

The frontend is a Next.js 16 application using the App Router with React 19.

6.1 Pages

Page	Path	Description
Home	/	Landing page with featured listings
Marketplace	/marketplace	Browse all listings with search and category filters
Listing Detail	/marketplace/[id]	View listing, trigger buy flow
Create Listing	/create	Upload product file and

Page	Path	Description
		preview image
Dashboard	/dashboard	Profile, listings, purchases, sales

6.2 Key Components

- **WalletConnect.js:** Handles MetaMask connection and SIWE sign-in flow
- **Navbar.js:** Navigation with wallet status and user pill
- **ListingCard.js:** Reusable listing preview card
- **AuthContext.js:** React context providing user, token, login, logout globally

6.3 Blockchain Integration

The buy flow in marketplace/[id]/page.js uses Wagmi hooks:

useWalletClient() → sends transactions via MetaMask
 useChainId() → detects current network
 useSwitchChain() → switches to Sepolia if needed

The purchase ID from the database is encoded as bytes32 using keccak256 from Viem, then passed to the smart contract's deposit() function along with the seller's wallet address and the listing price in ETH.

7. Database Design

The database uses PostgreSQL managed through Prisma ORM with three models.

7.1 User Model

Field	Type	Description
id	UUID	Primary key
walletAddress	String (unique)	Ethereum wallet address
username	String? (unique)	Optional display name
bio	String?	Profile bio
avatarUrl	String?	Profile image URL
createdAt	DateTime	Registration timestamp

7.2 Listing Model

Field	Type	Description
id	UUID	Primary key
title	String	Listing title

Field	Type	Description
description	String	Full description
price	Float	Price in ETH
fileUrl	String	Stored filename on disk
fileName	String?	Original filename shown to buyer
fileSize	Int?	File size in bytes
previewUrl	String?	Preview image path
category	String	software/art/music/ebook/template/other
tags	String[]	Array of tags
sellerId	String	FK → User
sellerAddress	String	Denormalized wallet address
sellerUsername	String?	Denormalized username
salesCount	Int	Total number of sales

7.3 Purchase Model

Field	Type	Description
id	UUID	Primary key (also used as escrow key)
buyerId	String	FK → User
buyerAddress	String	Denormalized buyer wallet
sellerId	String	FK → User
sellerAddress	String	Denormalized seller wallet
listingId	String	FK → Listing
listingTitle	String	Denormalized listing title
listingPrice	Float	Denormalized price at time of purchase
status	String	pending / confirmed / disputed
txHash	String?	On-chain transaction hash
confirmedAt	DateTime?	Delivery confirmation timestamp
disputedAt	DateTime?	Dispute timestamp

8. Authentication Flow

Assetry uses Sign-In with Ethereum (SIWE) — a standard for wallet-based authentication — combined with JWT for session management.

1. Client requests a nonce from the server
2. Client builds a SIWE message with the nonce, domain, and wallet address
3. User signs the message in MetaMask (no ETH spent — just a signature)
4. Client sends the signed message to the server
5. Server verifies the signature using the siwe library
6. If valid, server finds or creates the user in the database
7. Server returns a JWT token
8. Client stores the JWT in localStorage
9. All subsequent API requests include the JWT in the Authorization header

This approach means users never create passwords. Their Ethereum private key acts as their identity credential, and the signature proves ownership without exposing the key.

9. File Storage

Product files are stored on the server's local filesystem under `server/uploads/`. Multer handles multipart form uploads with the following configuration:

- **Product files:** Up to 100MB, supports ZIP, PDF, MP3, MP4, PNG, JPG, and other common digital asset formats
- **Preview images:** PNG, JPG, GIF, WebP only
- **Filename strategy:** Files are renamed to `{timestamp}-{random}{extension}` to prevent collisions and avoid exposing original filenames publicly

Preview images are served as static files via Express (`/uploads/filename`). Product files are only accessible through the authenticated download route.

10. Transaction Lifecycle

The complete lifecycle of a purchase on Assetry:

SELLER

- 1. Connects MetaMask → SIWE sign-in → JWT issued
- 2. Creates listing → uploads product file + preview image
 - POST /api/listings (multipart)
 - File saved to server/uploads/
 - Listing record created in PostgreSQL

BUYER

- 3. Connects MetaMask → SIWE sign-in → JWT issued
- 4. Browses marketplace → opens listing detail page
- 5. Clicks "Buy"
 - POST /api/purchases → purchase record **created** (status: pending)
 - purchaseId encoded as bytes32 via keccak256
 - MetaMask popup: **deposit**(purchaseId, sellerAddress) + ETH value
 - ETH locked in AssetryEscrow contract on Sepolia
 - txHash saved via PATCH /api/purchases/:id/tx
- 6. Buyer receives **asset** (downloads from seller or external delivery)
- 7. Buyer clicks "Confirm Delivery" in dashboard
 - MetaMask popup: **confirmDelivery**(purchaseId)
 - Smart contract releases ETH to seller's wallet
 - PATCH /api/purchases/:id/confirm → status: confirmed
 - Download button unlocked for buyer

OR

- 7b. Buyer clicks "Dispute" in dashboard
 - MetaMask popup: **refund**(purchaseId)
 - Smart contract returns ETH to buyers wallet
 - PATCH /api/purchases/:id/dispute → status: disputed

11. Workload Distribution

The project was developed collaboratively by three team members. The workload was divided as follows:

Muhammad Ali Jauhar— Backend & Database

- Designed the PostgreSQL database schema using Prisma ORM
- Built the Express REST API (auth, listings, purchases, users routes)
- Implemented SIWE authentication and JWT middleware
- Built the file upload system using Multer
- Implemented the protected file download route
- Managed database migrations

Muhammad Abdullah Khan Saadi— Blockchain & Smart Contract

- Designed and implemented the AssetryEscrow Solidity smart contract
- Set up Hardhat development environment and deployment pipeline
- Deployed the contract to Ethereum Sepolia testnet
- Integrated Wagmi and Viem hooks into the frontend for on-chain interactions
- Implemented the buy flow, confirm delivery, and refund flows on the frontend
- Wrote the escrow ABI and contract interaction utilities

Abdullah bin Umar— Frontend & UI

- Built all Next.js pages (home, marketplace, listing detail, create, dashboard)
- Designed and implemented the UI using custom CSS (design tokens, components)
- Built the WalletConnect, Navbar, and ListingCard components
- Implemented the AuthContext for global session management
- Built the dashboard (profile editing, listings management, purchases, sales)
- Integrated the API layer (lib/api.js) connecting frontend to backend

12. Conclusion

Assetry demonstrates a practical application of blockchain technology in solving a real-world trust problem in digital commerce. By combining a Solidity escrow smart contract with a full-stack web application, the platform achieves:

- **Trustless payments:** Neither party can manipulate the escrow — the smart contract enforces the rules
- **Decentralized identity:** Wallet-based authentication via SIWE eliminates passwords and centralized account management
- **Transparent transactions:** Every payment and release is recorded on the public Sepolia blockchain and verifiable by anyone

- **Practical usability:** The platform provides a familiar marketplace experience while abstracting blockchain complexity behind a clean UI

The two-transaction model (deposit on purchase, release on confirmation) is the core innovation — it mirrors the logic of a traditional escrow service but executes it autonomously through code, without requiring a trusted third party.